JAVA PROGRAMMING

Chapter 20: Collections Framework in Java

Personal Notes

Arrays are great for fixed-size data — but the moment you need to add, remove, search, or organize data dynamically, arrays start showing their limits. That's where Java's Collections Framework comes in.

The Collections Framework is a set of ready-made classes and interfaces for storing and manipulating groups of objects. It is one of the most-used parts of Java. Every real Java project — backend, Android, Spring, anywhere — uses it constantly.

1. What is the Collections Framework?

The Java Collections Framework is a unified architecture for representing and manipulating COLLECTIONS — groups of objects.

It provides three main things:

- Interfaces — abstract types like List, Set, Queue, Map
- Implementations — concrete classes like ArrayList, HashSet, HashMap
- Algorithms — utility methods like sort, search, shuffle, reverse

> **Quick mental model:** Interfaces define WHAT a collection can do. Implementations define HOW it does it. You write code against interfaces (List), and pick implementations (ArrayList) based on your performance needs.

2. Arrays vs Collections

If you already have arrays, why use collections? Arrays have real limitations:

Arrays	Collections
Fixed size — you must know the length up front	Dynamic size — grows and shrinks automatically
Only one data type per array	Can hold any objects (with generics for type safety)
Limited built-in operations	Rich set of built-in methods (sort, search, etc.)
No interfaces — can't swap implementations	Programming to interfaces gives flexibility
Primitive types allowed (int[], double[])	Only objects allowed (uses wrapper types: Integer, Double)
Fast and memory-efficient for fixed data	Slightly slower, but far more flexible

The simple rule: use arrays for fixed-size, performance-critical work. Use collections for everything else.

3. The Collection Hierarchy

Almost all collection classes implement one of four core interfaces. Knowing this picture saves a lot of confusion later.

```
                    Iterable<E>
                         |
                    Collection<E>              Map<K,V>     (NOT a
Collection!)
                   /     |     \                 |
                  /      |      \                +---------+----------+
                 v       v       v               |         |          |
             List<E>  Set<E>  Queue<E>         HashMap   LinkedHashMap
TreeMap
               |        |        |
       +-------+        +-------+----------+
       |        |       |       |          |
       v        v       v       v          v
   ArrayList  LinkedList HashSet TreeSet  PriorityQueue
              (also a    LinkedHashSet    ArrayDeque
               Queue)
```

The Four Core Categories

- List — ordered, allows duplicates, indexed access (like a smart array)
- Set — no duplicates, unordered (or ordered by sort) — like a math set
- Queue — FIFO order (first in, first out) — like a line at a counter
- Map — key-value pairs (not a Collection technically, but part of the framework)

> **Note about Map:** Map does NOT extend the Collection interface. It is its own thing — because key-value pairs don't fit the single-element model of Collections. But it's always discussed together since it's used the same way in real code.

4. The List Interface

A List is an ORDERED collection that allows DUPLICATES. Each element has an index (starting at 0). This is the most common collection you'll use.

4.1 ArrayList

Backed by a dynamic array. Fast for random access (by index), slower for inserting/removing in the middle.

```
import java.util.ArrayList;
import java.util.List;
```

```
public class Main {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Aman");          // [Aman]
        names.add("Riya");          // [Aman, Riya]
        names.add("Karan");         // [Aman, Riya, Karan]

        System.out.println(names.get(0));     // Aman
        System.out.println(names.size());     // 3
        System.out.println(names.contains("Riya"));   // true

        names.remove("Aman");                 // [Riya, Karan]
        names.set(0, "Priya");                // [Priya, Karan]

        for (String n : names) {
            System.out.println(n);
        }
    }
}
```

4.2 LinkedList

Backed by a doubly-linked list. Fast for inserting/removing anywhere, slower for random access by index. Implements both List AND Queue.

```
import java.util.LinkedList;

LinkedList<Integer> list = new LinkedList<>();
list.add(10);
list.add(20);
list.addFirst(5);     // [5, 10, 20]
list.addLast(30);     // [5, 10, 20, 30]

list.removeFirst();   // [10, 20, 30]
list.removeLast();    // [10, 20]

System.out.println(list);
```

4.3 ArrayList vs LinkedList

Aspect	ArrayList	LinkedList
Backed by	Dynamic array	Doubly linked list
get(index)	O(1) — instant	O(n) — walks the list
add at end	O(1) amortized	O(1)

Aspect	ArrayList	LinkedList
add at start	O(n) — shifts everything	O(1)
remove at middle	O(n)	O(n) — but no shifting
Memory	Less overhead per element	More (next/prev pointers)
Best for	Random access, mostly-read data	Frequent insert/remove at ends

> **Default choice:** Use ArrayList unless you have a specific reason not to. It's faster for most real workloads. Only switch to LinkedList when you do many insertions/removals at the beginning or middle.

5. The Set Interface

A Set is a collection that holds NO DUPLICATES. Each element appears at most once. Useful when you only care about whether something exists, not how many times.

5.1 HashSet

Backed by a hash table. Fastest set — but does not guarantee any order.

```
import java.util.HashSet;
import java.util.Set;

Set<String> unique = new HashSet<>();
unique.add("apple");
unique.add("banana");
unique.add("apple");        // ignored — duplicate

System.out.println(unique.size());           // 2
System.out.println(unique.contains("apple")); // true

unique.remove("banana");
System.out.println(unique);                  // [apple] (order not
guaranteed)
```

5.2 LinkedHashSet

Like HashSet but PRESERVES insertion order — slightly slower than HashSet but predictable iteration.

```
import java.util.LinkedHashSet;

LinkedHashSet<String> set = new LinkedHashSet<>();
set.add("banana");
set.add("apple");
```

```
set.add("cherry");
System.out.println(set);    // [banana, apple, cherry] — insertion order
preserved
```

5.3 TreeSet

Stores elements in SORTED order. Backed by a balanced tree. Slower than HashSet but always sorted.

```
import java.util.TreeSet;

TreeSet<Integer> sorted = new TreeSet<>();
sorted.add(30);
sorted.add(10);
sorted.add(20);

System.out.println(sorted);          // [10, 20, 30] — sorted
automatically
System.out.println(sorted.first());  // 10
System.out.println(sorted.last());   // 30
```

HashSet vs LinkedHashSet vs TreeSet

Class	Order	add/contains	Use when
HashSet	None	O(1) avg	When you just want uniqueness, fast
LinkedHashSet	Insertion order	O(1) avg	Unique + predictable iteration order
TreeSet	Sorted order	O(log n)	Unique + always sorted

6. The Queue Interface

A Queue holds elements in FIFO order (First In, First Out) — like people standing in a line at a counter. The first person to arrive is the first to be served.

6.1 LinkedList as Queue

```
import java.util.LinkedList;
import java.util.Queue;

Queue<String> q = new LinkedList<>();
q.offer("first");        // add to back
q.offer("second");
q.offer("third");

System.out.println(q.peek());    // first  (look at front)
```

```
System.out.println(q.poll());    // first  (remove from front)
System.out.println(q);           // [second, third]
```

6.2 PriorityQueue

A queue where the SMALLEST element comes out first (a min-heap by default). Not FIFO — order is by priority.

```
import java.util.PriorityQueue;

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);

System.out.println(pq.poll());   // 10  (smallest first)
System.out.println(pq.poll());   // 20
System.out.println(pq.poll());   // 30
```

6.3 ArrayDeque

A double-ended queue. You can add/remove from BOTH ends. Faster than LinkedList for queue use.

```
import java.util.ArrayDeque;

ArrayDeque<Integer> deque = new ArrayDeque<>();
deque.addFirst(1);
deque.addLast(2);
deque.addFirst(0);
// deque: [0, 1, 2]

deque.removeFirst();    // [1, 2]
deque.removeLast();     // [1]
```

> **Recommendation:** For most queue or stack work, use ArrayDeque. It's faster than LinkedList and replaces the old Stack class (which is obsolete). For priority-based work, use PriorityQueue.

7. The Map Interface

A Map stores KEY-VALUE pairs. Each key maps to one value. Keys must be unique; values can repeat. This is the most powerful and frequently used part of the framework.

7.1 HashMap

Backed by a hash table. Fastest map — but no order guarantee.

```
import java.util.HashMap;
import java.util.Map;
```

```
Map<String, Integer> scores = new HashMap<>();
scores.put("Aman", 92);
scores.put("Riya", 88);
scores.put("Karan", 95);

System.out.println(scores.get("Aman"));            // 92
System.out.println(scores.containsKey("Riya"));    // true
System.out.println(scores.size());                  // 3

scores.remove("Karan");

// Iterating
for (Map.Entry<String, Integer> entry : scores.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

7.2 LinkedHashMap

Like HashMap but PRESERVES insertion order during iteration.

```
import java.util.LinkedHashMap;

LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
map.put("B", 2);
map.put("A", 1);
map.put("C", 3);

for (String key : map.keySet()) {
    System.out.println(key);     // B, A, C  (insertion order)
}
```

7.3 TreeMap

Stores entries SORTED by key. Slower than HashMap, but always sorted.

```
import java.util.TreeMap;

TreeMap<String, Integer> map = new TreeMap<>();
map.put("Banana", 2);
map.put("Apple", 1);
map.put("Cherry", 3);

System.out.println(map);    // {Apple=1, Banana=2, Cherry=3}  (sorted by
key)
```

7.4 Useful Map Methods

- put(key, value) — add or update an entry

- get(key) — read the value for a key (returns null if not found)
- getOrDefault(key, defaultValue) — value or fallback
- containsKey(key) / containsValue(value)
- remove(key) — delete an entry
- keySet() — view of all keys
- values() — collection of all values
- entrySet() — set of all key-value pairs
- size() — total number of entries

7.5 The Frequency-Count Pattern

HashMap is the perfect tool for counting things. This is one of the most common interview patterns.

```
String s = "programming";
Map<Character, Integer> freq = new HashMap<>();

for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}

System.out.println(freq);
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

8. Iterating Over Collections

There are several ways to loop over collections. Pick whichever fits the situation.

8.1 Enhanced for-loop (for-each)

```
List<String> list = new ArrayList<>(Arrays.asList("a", "b", "c"));

for (String s : list) {
    System.out.println(s);
}
```

Cleanest and most common. Works on any Iterable. Cannot modify the collection during iteration.

8.2 Traditional indexed for-loop

```
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}
```

Use when you need the index. Works only for Lists, not Sets or Maps.

8.3 Iterator

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("b")) {
        it.remove();    // safe removal during iteration
    }
}
```

Use Iterator when you need to remove elements while iterating. Modifying a collection with a for-each loop throws ConcurrentModificationException.

8.4 forEach + Lambda (Java 8+)

```
list.forEach(s -> System.out.println(s));
list.forEach(System.out::println);     // method reference shortcut
```

9. Useful Methods Across Collections

Method	What it does
`add(e) / put(k,v)`	Add an element / key-value pair
`remove(e) / remove(k)`	Remove an element or entry
`contains(e) / containsKey(k)`	Check if an element or key exists
`size()`	Number of elements / entries
`isEmpty()`	True if no elements
`clear()`	Remove all elements
`iterator()`	Get an Iterator for safe removal during iteration
`forEach(action)`	Apply an action to every element (Java 8+)
`stream()`	Get a Stream for functional operations (Java 8+)
`Collections.sort(list)`	Sort a List in natural order
`Collections.reverse(list)`	Reverse a List in place
`Collections.max/min(coll)`	Find largest/smallest

Example: Sorting a List

```
List<Integer> nums = new ArrayList<>(Arrays.asList(5, 2, 8, 1, 9));

Collections.sort(nums);
System.out.println(nums);    // [1, 2, 5, 8, 9]

Collections.reverse(nums);
System.out.println(nums);    // [9, 8, 5, 2, 1]

// Custom sort using lambda
Collections.sort(nums, (a, b) -> b - a);   // descending
System.out.println(nums);
```

10. Time Complexity Cheat Sheet

This is the table interviewers love to ask about. Memorize the common operations.

Class	get / contains	remove	insert (middle)	add (end)
ArrayList	O(1)	O(n)	O(n)	O(1)*
LinkedList	O(n)	O(n)	O(1)	O(1)
HashSet	O(1) avg	O(1) avg	—	O(1) avg
LinkedHashSet	O(1) avg	O(1) avg	—	O(1) avg
TreeSet	O(log n)	O(log n)	—	O(log n)
HashMap	O(1) avg	O(1) avg	—	O(1) avg
TreeMap	O(log n)	O(log n)	—	O(log n)
PriorityQueue	O(n)	O(log n)	—	O(log n)

* ArrayList add at end is O(1) amortized (occasionally resizes).

11. When to Use Which Collection

If you need to...	Use
Need an ordered list, fast random access	**ArrayList**
Frequent insertions/removals at ends	**LinkedList or ArrayDeque**
Need uniqueness only, order doesn't matter	**HashSet**
Need uniqueness + insertion order	**LinkedHashSet**

If you need to...	Use
Need uniqueness + sorted order	**TreeSet**
Need key-value pairs, fast lookup	**HashMap**
Key-value + insertion order	**LinkedHashMap**
Key-value + sorted by key	**TreeMap**
Need FIFO (first-in-first-out) processing	**ArrayDeque (or LinkedList)**
Need priority-based ordering	**PriorityQueue**
Need a stack (LIFO)	**ArrayDeque (push/pop)**

12. Common Mistakes

Mistake 1: Using == to compare collections

```
List<Integer> a = Arrays.asList(1, 2, 3);
List<Integer> b = Arrays.asList(1, 2, 3);

a == b              // false — different objects
a.equals(b)         // true — same contents
```

Mistake 2: Modifying a collection during for-each iteration

```
for (String s : list) {
    if (s.equals("x")) {
        list.remove(s);    // ConcurrentModificationException!
    }
}

// Fix: use Iterator
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("x")) {
        it.remove();    // safe
    }
}
```

Mistake 3: Forgetting to override equals() and hashCode() for custom objects in HashSet/HashMap

If you put your own class objects into a HashSet or HashMap, you MUST override equals() and hashCode() — otherwise duplicates can appear and lookups will fail silently.

Mistake 4: Confusing List with array

```
int[] arr = new int[5];
arr.length          // ✓ property — no parentheses

List<Integer> list = new ArrayList<>();
list.size()         // ✓ method — has parentheses
list.length         // ✗ doesn't exist
```

Mistake 5: Using raw types instead of generics

```
List names = new ArrayList();      // raw — no type safety
List<String> names = new ArrayList<>();   // safe — only Strings allowed
```

Mistake 6: Picking the wrong collection for the job

Using LinkedList when you need random access by index is slow. Using ArrayList when you need lots of inserts at the start is slow. Always think about the operations you'll do MOST, then pick.

13. Best Practices

- Program to interfaces, not implementations: List<String> list = new ArrayList<>(); — easier to swap later
- Always use generics: List<String>, not raw List
- Default to ArrayList for lists, HashMap for maps, HashSet for sets
- Use Collections.unmodifiableList() or List.of() when you want read-only collections
- Use forEach with lambdas for clean iteration when you don't need indices
- Override equals() and hashCode() in your custom classes if they'll live in a Set or as Map keys
- Pick based on use case: read-heavy? ArrayList. Insert-heavy at ends? ArrayDeque.

14. Quick Reference

Interface / Class	Purpose	Common Implementations
List	Ordered, allows duplicates, indexed	ArrayList, LinkedList
Set	No duplicates	HashSet, LinkedHashSet, TreeSet
Queue	FIFO ordering	LinkedList, ArrayDeque, PriorityQueue
Map	Key-value pairs, unique keys	HashMap, LinkedHashMap, TreeMap
Iterator	Interface for safe iteration	Returned by iterator() method
Collections	Utility class with sort, reverse, max, etc.	Collections.sort(list)

Interface / Class	Purpose	Common Implementations
Arrays	Utility class for array operations	Arrays.asList(...), Arrays.sort(...)

15. Practice Problems

Try these to lock in the framework. Each one uses a different collection.

1. Create an ArrayList of 10 integers. Find the sum, max, and min using built-in or loops.
2. Take a sentence as input. Use a HashSet to print all unique words.
3. Take a string and count the frequency of each character using a HashMap.
4. Implement a simple to-do list using ArrayList. Support add, remove, and list operations.
5. Take 5 student names and marks. Store in a HashMap. Print the topper.
6. Use a TreeSet to maintain a sorted list of unique numbers as user inputs them.
7. Use a PriorityQueue to find the 3 smallest numbers from a list of 10.
8. Implement a stack using ArrayDeque. Push 5 numbers and pop them in reverse order.
9. Take two ArrayLists. Find their intersection (common elements) using a HashSet.
10. Take an array of words. Group them by their length using a HashMap<Integer, List<String>>.
11. Implement an LRU-style behavior: a LinkedHashMap that drops oldest when size exceeds 5.
12. Find the first non-repeating character in a string using LinkedHashMap (preserves order).

16. Final Takeaway

The Collections Framework is one of the most-used parts of Java. Master ArrayList, HashMap, HashSet, and ArrayDeque — these four cover 90% of real-world use. The rest you'll pick up naturally.

More important than memorizing every class is knowing WHICH ONE to reach for. That comes from understanding the trade-offs: ordered vs unordered, sorted vs insertion order, fast access vs fast insertion. Once those trade-offs click, picking the right collection becomes second nature.

> **Key Takeaway:** If you can confidently choose between ArrayList, HashMap, HashSet, and TreeMap based on the problem — and explain WHY — you're already ahead of most beginners. Collections aren't about memorization; they're about pattern recognition. Lots of duplicates? ArrayList. No duplicates? HashSet. Need a count? HashMap. Need sorted? TreeMap. That's 90% of it.

17. What's Next?

Now that collections feel natural, the next Java topics to explore are:

- Generics in depth — writing your own generic classes and methods
- Iterators and the Comparable / Comparator interfaces for custom sorting

- Java 8 Streams — functional-style operations on collections
- Lambda expressions and method references
- Optional class — handling "maybe null" values cleanly
- Concurrent collections (ConcurrentHashMap, etc.) for multithreaded code
- Exception handling — try, catch, finally, custom exceptions